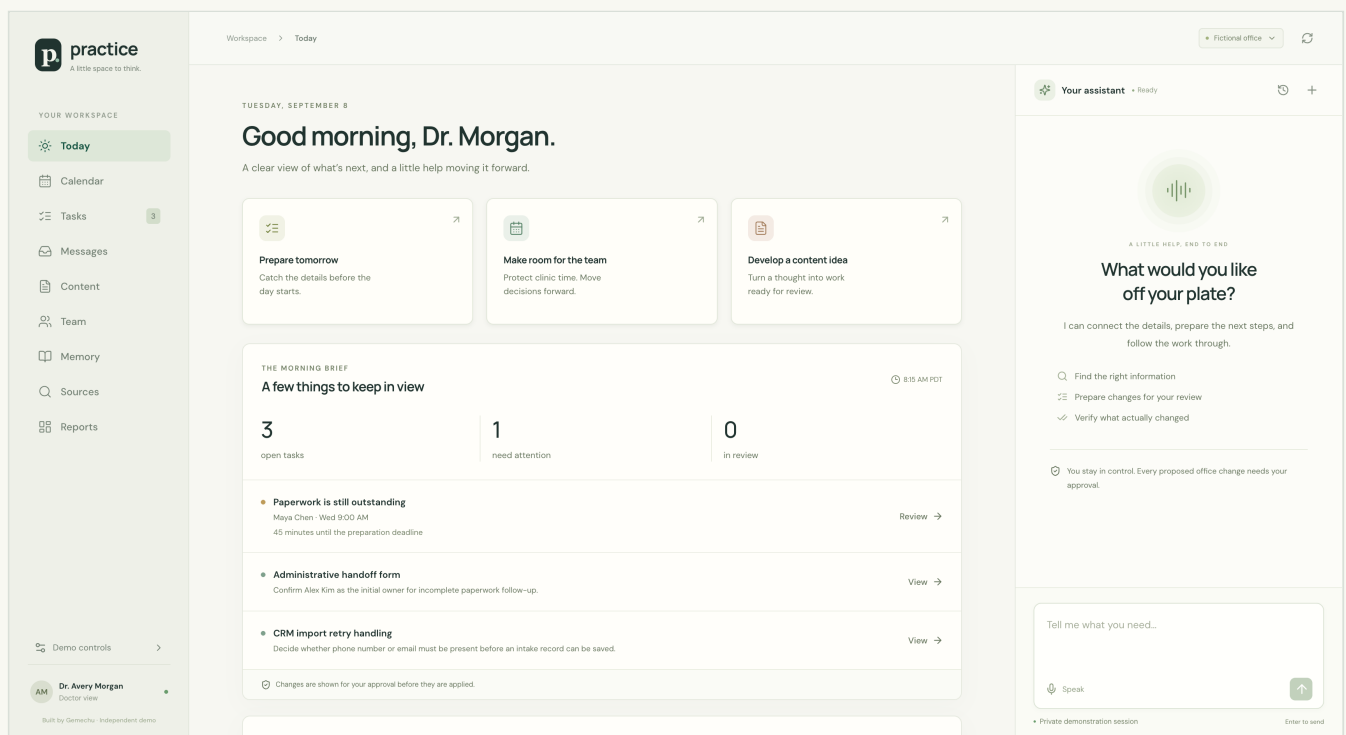


The 8:15 brief.

A practice assistant that can show its work.

I built a working demonstration around the moment a physician needs to turn scattered office information into a few clear, reviewable actions.



Actual hosted interface with Supabase anonymous sign-in. Dr. Avery Morgan and every office record are fictional.

01 Prepare tomorrow

Read the schedule, calculate paperwork deadlines and coordinate the existing follow-up.

02 Make room for the team

Inspect constraints, propose a specific calendar change and re-check it before writing.

03 Develop a content idea

Retrieve public sources, draft a script and caption, and propose review and recording work.

Built by Gemechu Taye

[Open the demonstration](#)

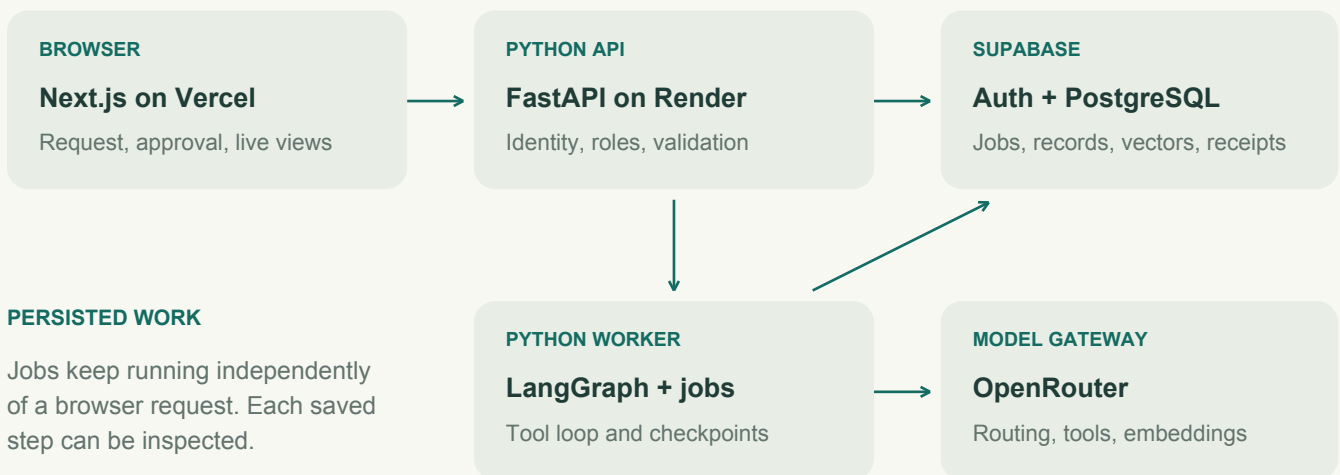
[Read the code](#)

AI-assisted implementation. Independent of Dr. Emer's practice and EmerGPT; no private practice systems are connected.

An assistant layer around existing systems.

The job calls for a personal assistant across calendars, communications, dashboards, knowledge and engineering work. Public research led me to focus on coordinating physician work around software the practice already describes using.

The EmerGPT announcement describes specialist-oriented AI [1]. Krazimo describes a practice platform and concierge with Salesforce and ModMed connections [2]. These are public accounts, not verification of the current installation. My design inference: demonstrate a physician-facing assistant that coordinates work across existing sources while preserving their ownership.



Supabase also supplies private evidence storage and owner-scoped change notifications. A real MCP server exposes scoped tools; a versioned demonstration handoff accepts synthetic paperwork and review events. Neither interface claims compatibility with an undocumented EmerGPT API.

The model chooses work. The application authorizes it.

Identity comes from the validated session. Roles and record versions are checked in Python. The model can propose a change, but it cannot approve its own plan or expand its own permissions.

Deployment status is recorded in the accompanying access guide. Architecture alone is not evidence that every hosted process is running.

Approval is a stored agreement.

A fluent answer is easy to produce. The harder question is whether the exact work the person reviewed is still valid when the assistant is ready to perform it.

1 READ

Current records, versions and related tasks.

2 PROPOSE

Exact payloads, stable IDs and an expiry.

3 APPROVE

The user approves that stored plan.

4 VERIFY

Re-check, write and read back receipts.

What the stored plan protects

- The action payload and approval expiry are fixed. A later record or source change can make approval stale.
- Calendar checks include conflicts, protected patient appointments and relevant team constraints. A changed office requires a fresh proposal.
- A write and its receipt commit together. A retry checks that receipt before attempting the action again.
- A job lease has a generation number. A worker that lost ownership cannot continue writing or reporting success.

A concrete administrative flow

The seeded office opens at 8:15 a.m. with a 9:00 a.m. appointment tomorrow. The public financial-policy note supplies a 24-hour paperwork rule [3], so code calculates 45 minutes remaining. The assistant reads the existing task before proposing follow-up; approved delivery goes only to the demonstration inbox.

When the synthetic "Paperwork arrives" event is applied, the linked, authorized preparation task closes. Unrelated tasks remain unchanged. A duplicate event does not repeat the effect.

A plan can finish partially.

Actions commit individually. The interface distinguishes completed work, unfinished work and stale work; it does not claim an all-or-nothing transaction across multiple actions.

Private context stays out of public drafts.

The content specialist receives selected public source notes and fixed writing instructions. It does not receive the doctor's full conversation, patient administration records or a free-form private briefing.

Inside the office

Calendar, task ownership, administrative preparation, private messages, engineering decisions and confirmed preferences are scoped to a workspace and role.

Public content specialist

Source IDs, attributed excerpts, a format and an angle. The specialist returns an unsaved draft. Saving content and creating review work still require approval.

Retrieval is scoped before ranking

The source store combines PostgreSQL full-text ranking with pgvector similarity. Embeddings have 1,536 dimensions and are refreshed when the source-note hash or embedding model changes. Workspace and role filters apply before candidates are ranked or snippets reach a model.

Each result retains its title, URL, access date and public-versus-demonstration label. A URL recheck records reachability and a content fingerprint; it does not silently rewrite the reviewed note or certify that a price or policy is current.

Memory has an owner and a scope

Confirmed preferences are persisted as versioned records. The interface supports inspection, editing and deletion. A preference can guide an answer; it cannot grant permission. Doctor, coordinator and editor views expose different records within the owner's fictional office.

Content has a reviewable result

A verified browser flow retrieved public notes [4], produced a sourced script and caption, and saved approved content with a review task and recording work. A later synthetic review event closed only the linked review task. Public material is drafted for review; nothing is posted to a real social account.

Voice and observable model calls

Hosted speech services were exercised with synthetic audio: Whisper returned the expected transcript, and Kokoro speech was transcribed back correctly. An in-process OpenTelemetry exporter captured a real model span; an external collector was not exercised.

Measured behavior, with the limits visible.

The verification suite records **118 passing automated tests**. Separate artifacts capture real model requests, a physical executor crash, browser workflows and hosted account isolation. Each piece answers a different question.

Check	Observed result
Physical process crash	In an isolated local PostgreSQL test, an executor was killed with SIGKILL after one commit. A replacement acquired lease generation 2 and completed the plan. No model calls were made.
Duplicate prevention	Recovery left exactly one task, one demonstration message and two receipts. The first completed action was reconciled, not repeated.
Changed constraints	An occupied proposed slot made the earlier approval stale. The interface requested a fresh approval.
Hosted isolation	Two anonymous sessions saw only their own workspaces. Direct office-record reads were blocked. WebSocket updates respected ownership.
Browser state	Local browser flows used live models and PostgreSQL for preparation, content and recovery. The hosted interface separately passed view, identity and isolation checks.

A small routing probe, not a benchmark

Three requests were sent to both a fast classifier and the planning model. All six returned the expected route. The recorded wall times below include the provider request; they do not measure full agent workflows.

Expected route	Gemini Flash Lite	GPT Terra
Content	1.05s / \$0.000043	4.38s / \$0.000534
Report	0.85s / \$0.000042	5.70s / \$0.000260
Planner	0.84s / \$0.000043	1.63s / \$0.000270

Three paired samples; different token counts. These observations justify inspecting routing cost, not a general speed, quality or savings claim. Requested model IDs: google/gemini-3.5-flash-lite and openai/gpt-5.6-terra.

A walkthrough I can defend.

I used AI-assisted implementation, including parallel specialist reviews and automated verification. The repository, action records and provider traces are available for inspection. My walkthrough focuses on the architecture, its failure cases and the specific code that enforces its rules.

In the walkthrough	What to inspect
1. Ask a fresh question	Use different wording. Follow model-selected tools through actual record reads, source retrieval and the proposed action payload.
2. Change the office	Occupy a proposed slot or complete paperwork. Watch the assistant distinguish stale work from already-completed work.
3. Interrupt execution	Inspect the stored receipt and replacement lease. Resume without duplicating the committed task.
4. Follow the boundary	Compare doctor and editor views, inspect a public-content prompt, then open the signed evidence export.

Where the important behavior lives

agent.py: model-selected tools and checkpoints. **domain.py**: approval validation and action receipts. **jobs.py** / **worker.py**: durable jobs, leases and recovery. **knowledge.py**: scoped text/vector retrieval. **auth.py**: identity validation. **apps/web**: the review and operator experience.

The implemented stack covers Python, agent tools, persistent memory, RAG, model routing, operational views, content review, MCP, APIs, PostgreSQL, cloud deployment and observable recovery. CRM/EMR records and incoming events are fictional adapters. No private EmerGPT, calendar, email or clinical-system connection is claimed.

[Try Practice Assistant](#)

[View the source and evidence](#)

This demonstrates engineering behavior, not HIPAA compliance, clinical validation or measured practice outcomes. Connecting a real practice would require approved system access, staff identity, vendor agreements and validation against its actual workflows.

Public sources that shaped the demonstration

[1] [EmerGPT public announcement](#)

[2] [Krazimo practice-platform case study](#)

[3] [Practice financial policy](#)

[4] [Practice scar-revision information](#)

Public sources reviewed September 8, 2026. Vendor and announcement claims are attributed, not independently verified. This project does not reproduce their private systems.